

Torc: Durable and Resource-Aware Workflow Orchestration from Workstations to Slurm Clusters

Daniel Thom^{1¶}, Nathan Keilbart², Joseph McKinsey¹, Pedro Andres Sanchez Perez¹, Andrew Lai², and Kapil Duwadi³

¹ National Laboratory of the Rockies ² Lawrence Livermore National Laboratory ³ Pacific Northwest National Laboratory ¶ Corresponding author

DOI: [N/A](#)

Software

- [Review](#) ↗
- [Repository](#) ↗
- [Archive](#) ↗

Editor: [Open Journals](#) ↗

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: 01 January 1970

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Torc is an open-source workflow orchestration system for command-oriented computational pipelines. It supports workflows ranging from independent parameter sweeps to dependency-rich, multi-stage campaigns with heterogeneous CPU, memory, GPU, node-count, and runtime requirements. Researchers can describe workflows declaratively, run them locally, and execute the same workflow through workers inside Slurm allocations. A REST service and SQLite database maintain durable workflow state, while a unified command-line interface supports creation, execution, monitoring, diagnosis, resource-usage analysis, selective reruns, and recovery. Python and generated API clients provide programmatic access, while terminal and web interfaces support interactive operation. Torc is intended for computational scientists who need a practical path from workstation-scale experiments to persistent high-performance computing campaigns.

Statement of need

Computational studies commonly begin with shell scripts, local process pools, or scheduler job arrays. These approaches can be sufficient for independent tasks, but scientific campaigns often grow to include dependencies, heterogeneous resource requirements, long execution histories, and failures that must be diagnosed and selectively rerun. Moving such a campaign to a high-performance computing system can force researchers to change how they describe work, manage state, submit jobs, inspect progress, and recover from failures.

Torc addresses this transition with a common workflow and control model across local and Slurm execution. Workflow definitions describe commands, dependencies, files, user data, resource requirements, schedulers, and failure behavior. Torc persists this information and execution state behind an HTTP API. Workers claim ready jobs according to available resources, execute them as local processes or Slurm job steps, and return results and resource observations. This separation allows the same workflow to be inspected and operated through a command-line interface, Python client, terminal interface, web dashboard, or direct API calls.

The compact control plane is important to Torc's intended use. Its server uses SQLite and does not require researchers to administer a separate database service for core workflow state. Torc is not intended to replace every workflow language, distributed runtime, or high-performance computing scheduler. It targets command-oriented scientific campaigns that benefit from durable coordination, resource-aware execution, and a consistent path between workstations and Slurm clusters.

State of the field

Scientific workflow systems span several overlapping design spaces. Snakemake and Nextflow provide mature dataflow-oriented workflow languages and broad execution portability (Di Tommaso et al., 2017; Mölder et al., 2021). Pegasus provides planning, provenance, data management, and recovery for large scientific workflows (Deelman et al., 2019). Parsl provides Python-native parallel scripting and high-performance computing executors (Babuji et al., 2019). FireWorks and Balsam provide persistent services and worker-based execution for high-throughput scientific campaigns (Jain et al., 2015; Salim et al., 2018). Pilot systems and hierarchical schedulers, including RADICAL-Pilot and Flux, execute many tasks within larger resource allocations (Ahn et al., 2020; Merzky et al., 2019).

Torc does not claim to originate directed acyclic graph execution, data-derived dependencies, persistent workflow state, dynamic workflows, or task execution inside allocations. It was built to provide a different integrated design point: command-oriented workflows, a compact control plane that does not require an external database service, and one operational model from local execution through Slurm. Independent workers and allocations pull from the same durable queue, while measured outcomes feed monitoring, selective reruns, and resource-aware recovery. This combination addresses campaigns that have outgrown scripts and job arrays but do not require a specialized distributed runtime or a separately administered workflow service.

This position differs from file-centered systems by allowing explicit dependencies and database-backed JSON relationships to coexist with file dataflow. It differs from Python-native runtimes by keeping existing executables and shell commands as the unit of work, while still providing Python and generated clients for orchestration. Compared with pilot and hierarchical schedulers, Torc adds the persistent workflow graph, user-facing lifecycle operations, and execution evidence around allocation-resident work. The contribution is therefore not any isolated primitive, but their coupling into a small operational system intended for scientific teams to run themselves.

Software design

Torc uses a client-server architecture. An asynchronous Rust server exposes a versioned REST API and stores workflows, jobs, dependencies, files, user data, resource requirements, results, events, and scheduler records in SQLite. Clients and workers communicate with the server over HTTP, allowing the database to remain on server-local storage while workers execute on other nodes. This design trades the horizontal write scalability of a distributed database for a control plane that is simple to deploy, back up, and inspect.

SQLite is a deliberate operating boundary rather than an interchangeable implementation detail. Only the server opens the database; remote workers use HTTP, so a live database need not reside on a parallel filesystem or be exposed to compute nodes. A campaign can be started without provisioning a database service, resumed after process restarts, and archived by preserving one file. This favors the common case of one coordinating server and many execution workers. Workloads that exceed a single server's write capacity would require a different persistence architecture, a trade-off Torc makes in favor of deployability on institutional and leadership-class computing systems.

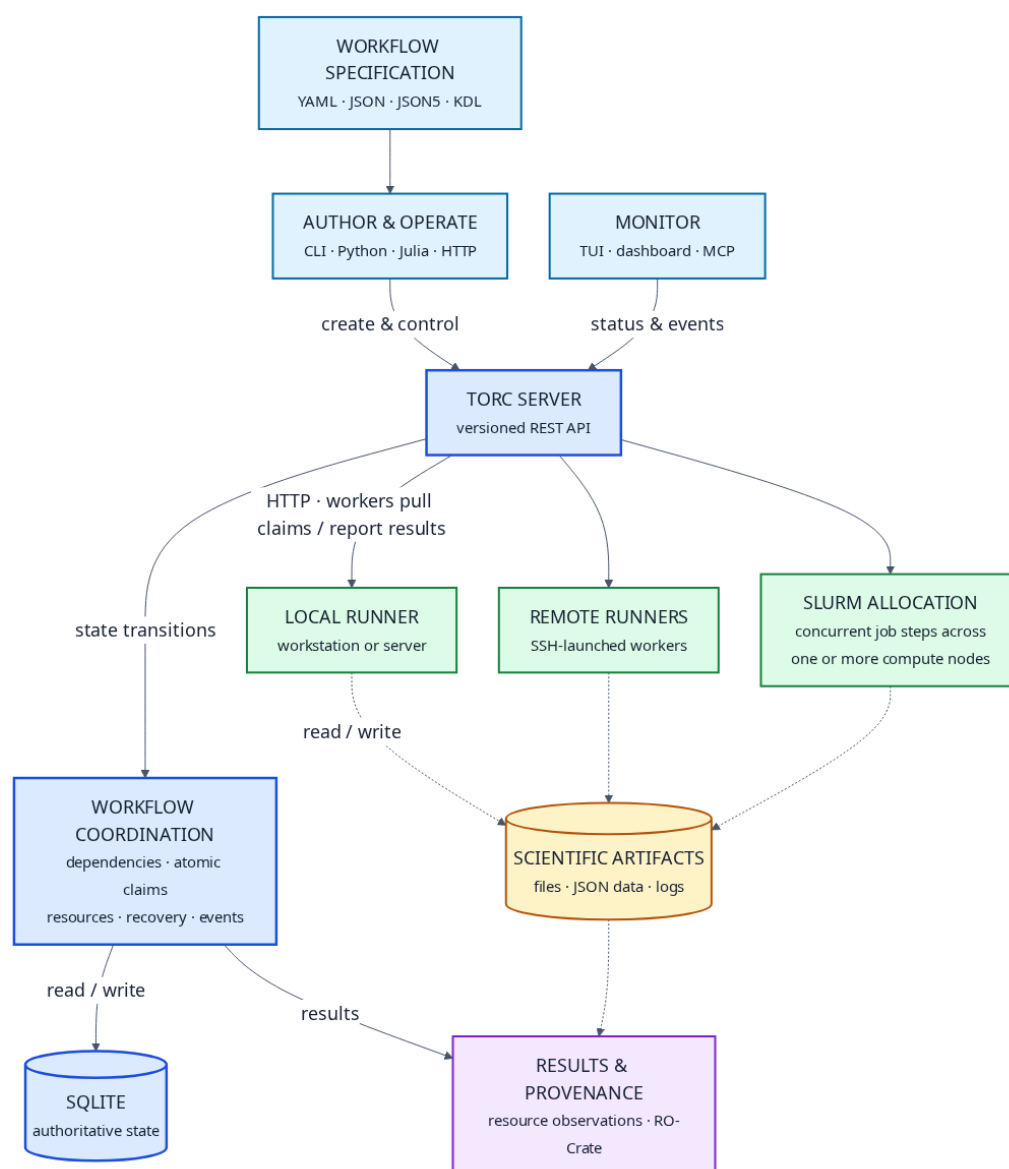


Figure 1: Torc separates user-facing tools, a durable SQLite-backed control plane, and pull-based execution across local, remote, and Slurm resources. Solid lines show control traffic; dashed lines show scientific artifact access.

Dependencies may be declared directly or inferred from producer and consumer relationships over files and JSON user data. Torc resolves these relationships into one graph so that the same state transitions apply regardless of how an edge was specified. A worker advertises its available CPUs, memory, GPUs, node count, and remaining runtime. The server selects fitting ready jobs and marks them pending in one transaction, preventing duplicate ownership when many workers request work concurrently. Priority ordering and residual backfill let smaller jobs use capacity left by larger claims. This is useful for heterogeneous campaigns in which several independently submitted Slurm allocations, each with its own runner by default, draw from the same workflow queue.

Runners separate workflow coordination from process placement. Local and remote runners execute commands directly. Inside a Slurm allocation, a runner can execute commands directly or launch resource-constrained `srun --exact` job steps; one runner can manage one

or more allocated nodes. Slurm therefore retains responsibility for allocations, placement, and enforcement, while Torc decides which dependency-ready job fits next. Researchers can develop a workflow on a workstation and move it to a cluster without changing its dependency or operational model.

For adaptive algorithms, transactional spawning adds a batch of jobs, parent and explicit dependency edges, and lineage state together. Invalid batches are rolled back. An iterative method can therefore inspect an intermediate result, create only its next generation, and terminate by spawning nothing when convergence is reached. Lineage records identify each generation, while iteration limits and replay handling constrain runaway or repeated requests. For completed workflows, reinitialization detects changed files, user data, job definitions, or missing outputs and resets only affected jobs and their descendants. Both mechanisms avoid predeclaring unnecessary work or rerunning unaffected branches.

Execution results include status, logs, attempts, and, when monitoring or scheduler accounting is available, CPU, memory, and runtime observations. These records support reports and resource plots, but they also close the control loop: likely memory and runtime failures can be diagnosed, resource requirements corrected, and selected work retried in replacement allocations. If the API becomes temporarily unavailable, workers can journal completions locally and reconcile them later rather than discard finished work or rerun expensive calculations. The same retained evidence lets users compare requested and observed resources after a campaign and tune later runs, even when automatic recovery is not used. Optional RO-Crate generation records workflow, job, input, output, and software provenance (Soiland-Reyes et al., 2022).

Users interact with these capabilities through declarative workflow files and a unified torc command-line interface. The same API supports generated clients, a Python orchestration layer, a terminal interface, a web dashboard, and an MCP server. This layered design allows interactive and programmatic clients to share workflow semantics rather than independently implementing workflow state transitions. It also exposes live status and historical resource evidence without requiring a separate monitoring stack.

Research impact statement

At the National Energy Research Scientific Computing Center at Lawrence Berkeley National Laboratory, Torc orchestrated ExaEpi parameter sweeps for predicting epidemiological events, including COVID-19. The campaign ran three million simulations in batches of 100,000 on the Perlmutter system. Its Slurm workflow combined GPU-backed main simulations with CPU-backed setup and postprocessing jobs. Resource-aware scheduling reduced required walltime relative to the prior configuration and limited unused allocation capacity between the heterogeneous job stages.

At Lawrence Livermore National Laboratory's Livermore Computing Dane system, Torc ran year-long power-system production-cost simulations across varied input scenarios. These CPU-only workloads were longer lived than the ExaEpi jobs. Resource monitoring exposed CPU and memory use for debugging and allowed the team to tune resource requests for subsequent simulations.

Availability

Torc is distributed under the BSD 3-Clause license. Source code, documentation, examples, integration tests, and release artifacts are available at <https://github.com/NatLabRockies/torc>. The project uses automated tests and generated-client consistency checks across its Rust server and clients. The version reviewed with this paper will be archived with a persistent DOI, and citation metadata will be provided in the repository.

AI usage disclosure

OpenAI GPT-5.6, accessed through OpenCode, was used for literature discovery, manuscript brainstorming, drafting, and formatting this paper for JOSS. Before submission, the authors will review and edit all AI-assisted text and verify its technical claims, references, and comparisons against the software, primary literature, and released evidence. The authors made the software's core design decisions and retain responsibility for the final manuscript.

Acknowledgements

Acknowledgements and funding information will be added before submission.

References

- Ahn, D. H., Bass, N., Chu, A., Garlick, J., Grondona, M., Lipari, D., Martin, B., Stearman, M., Springmeyer, B., & Yang, T. (2020). Flux: Overcoming scheduling challenges for exascale workflows. *Future Generation Computer Systems*, 110, 202–213. <https://doi.org/10.1016/j.future.2020.04.006>
- Babuji, Y., Woodard, A., Li, Z., Katz, D. S., Clifford, B., Kumar, R., Lacinski, L., Chard, R., Wozniak, J. M., Foster, I., Wilde, M., & Chard, K. (2019). Parsl: Pervasive parallel programming in Python. *Proceedings of the 28th International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*, 25–36. <https://doi.org/10.1145/3307681.3325400>
- Deelman, E., Vahi, K., Rynge, M., Mayani, R., Silva, R. F. da, Papadimitriou, G., & Livny, M. (2019). The evolution of the Pegasus workflow management software. *Computing in Science & Engineering*, 21(4), 22–36. <https://doi.org/10.1109/MCSE.2019.2919690>
- Di Tommaso, P., Chatzou, M., Floden, E. W., Barja, P. P., Palumbo, E., & Notredame, C. (2017). Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35(4), 316–319. <https://doi.org/10.1038/nbt.3820>
- Jain, A., Ong, S. P., Chen, W., Medasani, B., Qu, X., Kocher, M., Brafman, M., Petretto, G., Rignanese, G.-M., Hautier, G., Gunter, D., & Persson, K. A. (2015). FireWorks: A dynamic workflow system designed for high-throughput applications. *Concurrency and Computation: Practice and Experience*, 27(17), 5037–5059. <https://doi.org/10.1002/cpe.3505>
- Merzky, A., Turilli, M., Maldonado, M., Santcroos, M., & Jha, S. (2019). Using pilot systems to execute many-task workloads on supercomputers. *Job Scheduling Strategies for Parallel Processing*, 61–82. https://doi.org/10.1007/978-3-030-10632-4_4
- Mölder, F., Jablonski, K. P., Letcher, B., Hall, M. B., Tomkins-Tinch, C. H., Sochat, V., Forster, J., Lee, S., Twardziok, S. O., Kanitz, A., Wilm, A., Holtgrewe, M., Rahmann, S., Nahnsen, S., & Köster, J. (2021). Sustainable data analysis with Snakemake. *F1000Research*, 10, 33. <https://doi.org/10.12688/f1000research.29032.2>
- Salim, M. A., Uram, T. D., Childers, J. T., Balaprakash, P., Vishwanath, V., & Papka, M. E. (2018). Balsam: Automated scheduling and execution of dynamic, data-intensive HPC workflows. *Proceedings of the 8th Workshop on Python for High-Performance and Scientific Computing*.
- Soiland-Reyes, S., Sefton, P., Crosas, M., Castro, L. J., Coppens, F., Fernández, J. M., Garijo, D., Grüning, B., La Rosa, M., Leo, S., Ó Carragáin, E., Portier, M., Trisovic, A., RO-Crate Community, Groth, P., & Goble, C. (2022). Packaging research artefacts with RO-Crate. *Data Science*, 5(2), 97–138. <https://doi.org/10.3233/DS-210053>